// Security Assessment

10.27.2025 - 10.27.2025

M Portal Lite

M0

HALBORN

M Portal Lite - M0

Prepared by: HALBORN

Last Updated 11/12/2025

Date of Engagement: October 27th, 2025 - October 27th, 2025

Summary

100% OF ALL REPORTED FINDINGS HAVE BEEN ADDRESSED

ALL FINDINGS	CRITICAL	HIGH	MEDIUM	LOW	INFORMATIONAL
2	0	0	0	0	2

TABLE OF CONTENTS

1. Introduction

`M0` engaged Halborn to conduct a security assessment of their smart contracts on October 27th, 2025. The scope of this assessment was limited to the smart contracts provided to the Halborn team. Commit hashes and additional details are documented in the Scope section of this report.

2. Assessment Summary

The `Halborn` team dedicated 1 day to this engagement, deploying one full-time security engineer to evaluate the smart contracts' security posture. The assigned security engineer is an expert in blockchain and smart contract security, with advanced skills in penetration testing, smart contract exploitation, and a comprehensive understanding of multiple blockchain protocols.

The objectives of this assessment were to:

- Verify that the smart contract functions operate as intended.
- Identify potential security vulnerabilities within the smart contracts.

In summary, Halborn identified few areas for improvement to reduce both the likelihood and impact of potential risks, which were partially addressed by the `M0 team`. The primary suggestions included:

- `Pack storage variables to improve gas efficiency.`
- `Snapshot the wiped-off liabilities before setting them to zero.`

3. Test Approach And Methodology

Halborn employed a combination of manual, semi-automated, and automated security testing methods to ensure effectiveness, efficiency, and accuracy within the scope of this assessment. Manual testing was vital for uncovering issues related to logic, processes, and implementation details, while automated techniques enhanced code coverage and helped identify deviations from security best practices. The assessment involved the following phases and tools:

- Research into the architecture and purpose of the smart contracts.
- Manual review and walkthrough of the smart contract code.
- Manual evaluation of critical Solidity variables and functions to identify potential vulnerability classes.
- Manual testing using custom scripts.
- Static security analysis of the scoped contracts and imported functions utilizing `Slither`.
- Local deployment and testing with `Foundry`.

4. RISK METHODOLOGY

Every vulnerability and issue observed by Halborn is ranked based on **two sets** of **Metrics** and a **Severity Coefficient**. This system is inspired by the industry standard Common Vulnerability Scoring System.

The two **Metric sets** are: **Exploitability** and **Impact**. **Exploitability** captures the ease and technical means by which vulnerabilities can be exploited and **Impact** describes the consequences of a successful exploit.

The **Severity Coefficients** is designed to further refine the accuracy of the ranking with two factors: **Reversibility** and **Scope**. These capture the impact of the vulnerability on the environment as well as the number of users and smart contracts affected.

The final score is a value between 0-10 rounded up to 1 decimal place and 10 corresponding to the highest security risk. This provides an objective and accurate rating of the severity of security vulnerabilities in smart contracts.

The system is designed to assist in identifying and prioritizing vulnerabilities based on their level of risk to address the most critical issues in a timely manner.

4.1 EXPLOITABILITY

ATTACK ORIGIN (AO):

Captures whether the attack requires compromising a specific account.

ATTACK COST (AC):

Captures the cost of exploiting the vulnerability incurred by the attacker relative to sending a single transaction on the relevant blockchain. Includes but is not limited to financial and computational cost.

ATTACK COMPLEXITY (AX):

Describes the conditions beyond the attacker's control that must exist in order to exploit the vulnerability. Includes but is not limited to macro situation, available third-party liquidity and regulatory challenges.

METRICS:

EXPLOITABILITY METRIC (M_E)	METRIC VALUE	NUMERICAL VALUE
Attack Origin (AO)	Arbitrary (AO:A) Specific (AO:S)	1 0.2

EXPLOITABILITY METRIC (M_E)	METRIC VALUE	NUMERICAL VALUE
Attack Cost (AC)	Low (AC:L) Medium (AC:M) High (AC:H)	1 0.67 0.33
Attack Complexity (AX)	Low (AX:L) Medium (AX:M) High (AX:H)	1 0.67 0.33

Exploitability E is calculated using the following formula:

$$E = \prod m_e$$

4.2 IMPACT

CONFIDENTIALITY (C):

Measures the impact to the confidentiality of the information resources managed by the contract due to a successfully exploited vulnerability. Confidentiality refers to limiting access to authorized users only.

INTEGRITY (I):

Measures the impact to integrity of a successfully exploited vulnerability. Integrity refers to the trustworthiness and veracity of data stored and/or processed on-chain. Integrity impact directly affecting Deposit or Yield records is excluded.

AVAILABILITY (A):

Measures the impact to the availability of the impacted component resulting from a successfully exploited vulnerability. This metric refers to smart contract features and functionality, not state. Availability impact directly affecting Deposit or Yield is excluded.

DEPOSIT (D):

Measures the impact to the deposits made to the contract by either users or owners.

YIELD (Y):

Measures the impact to the yield generated by the contract for either users or owners.

METRICS:

IMPACT METRIC (M_I)	METRIC VALUE	NUMERICAL VALUE
Confidentiality (C)	None (C:N) Low (C:L) Medium (C:M) High (C:H) Critical (C:C)	0 0.25 0.5 0.75 1
Integrity (I)	None (I:N) Low (I:L) Medium (I:M) High (I:H) Critical (I:C)	0 0.25 0.5 0.75 1
Availability (A)	None (A:N) Low (A:L) Medium (A:M) High (A:H) Critical (A:C)	0 0.25 0.5 0.75 1
Deposit (D)	None (D:N) Low (D:L) Medium (D:M) High (D:H) Critical (D:C)	0 0.25 0.5 0.75 1
Yield (Y)	None (Y:N) Low (Y:L) Medium (Y:M) High (Y:H) Critical (Y:C)	0 0.25 0.5 0.75 1

Impact I is calculated using the following formula:

$$I = max(m_I) + \frac{\sum m_I - max(m_I)}{4}$$

4.3 SEVERITY COEFFICIENT

REVERSIBILITY (R):

Describes the share of the exploited vulnerability effects that can be reversed. For upgradeable contracts, assume the contract private key is available.

SCOPE (S):

Captures whether a vulnerability in one vulnerable contract impacts resources in other contracts.

METRICS:

SEVERITY COEFFICIENT (C)	COEFFICIENT VALUE	NUMERICAL VALUE
Reversibility (r)	None (R:N) Partial (R:P) Full (R:F)	1 0.5 0.25
Scope (s)	Changed (S:C) Unchanged (S:U)	1.25 1

Severity Coefficient C is obtained by the following product:

$$C = rs$$

The Vulnerability Severity Score S is obtained by:

$$S = min(10, EIC * 10)$$

The score is rounded up to 1 decimal places.

SEVERITY	SCORE VALUE RANGE
Critical	9 - 10
High	7 - 8.9
Medium	4.5 - 6.9
Low	2 - 4.4

SEVERITY	SCORE VALUE RANGE
Informational	0 - 1.9

5. SCOPE

REPOSITORY

(a) Repository: m-portal-lite

(b) Assessed Commit ID: d48135f

(c) Items in scope:

- src/HubPortal.sol
- src/Portal.sol
- src/SpokePortal.sol
- **m0-foundation/m-portal-lite/pull/39**

Out-of-Scope: Third party dependencies and economic attacks.

REMEDIATION COMMIT ID:

- f59bdea

Out-of-Scope: New features/implementations after the remediation commit IDs.

6. ASSESSMENT SUMMARY & FINDINGS OVERVIEW

CRITICAL	HIGH	MEDIUM	LOW	INFORMATIONAL
0	0	0	0	2

SECURITY ANALYSIS	RISK LEVEL	REMEDIATION DATE
GAS INEFFICIENCY FROM UNPACKED STORAGE VARIABLES	INFORMATIONAL	ACKNOWLEDGED - 11/01/2025

SECURITY ANALYSIS	RISK LEVEL	REMEDIATION DATE
UNSAFE PRINCIPAL RESET BY OWNER MAY LEAD TO APPARENT INSOLVENCY	INFORMATIONAL	SOLVED - 10/31/2025

7. FINDINGS & TECH DETAILS

7.1 GAS INEFFICIENCY FROM UNPACKED STORAGE VARIABLES

// INFORMATIONAL

Description

In `HubPortal.sol`, both `_burnOrLock` and `_mintOrUnlock` read two distinct storage mappings on each invocation: `crossSpokeConnectionEnabled[spokeChainId_]` and `bridgedPrincipal[spokeChainId_]`. Each mapping access incurs a separate SLOAD operation.

Increased gas consumption is incurred for every bridging operation involving an "Isolated" spoke, and accumulated additional gas costs are expected to grow over the protocol's lifetime.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N (0.0)

Recommendation

It is recommended that these two variables be packed into a struct. If packed into a struct, the two SLOAD operations would be merged into one, yielding a significant gas reduction for each core-bridging transaction.

Remediation Comment

ACKNOWLEDGED: The **M0 team** acknowledged this finding.

7.2 UNSAFE PRINCIPAL RESET BY OWNER MAY LEAD TO APPARENT INSOLVENCY

// INFORMATIONAL

Description

The `enableCrossSpokeConnection` function, located in `HubPortal.sol`, is used by the contract owner to change a spoke's state from "Isolated" to "Connected". During this transition, the spoke's `bridgedPrincipal` value is unconditionally reset to zero.

A discrepancy is created in the Hub's on-chain accounting: recorded liabilities are left not to reflect the actual outstanding principal. Consequently, the protocol's apparent solvency on-chain is reduced by the amount of the reset principal.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N (0.0)

Recommendation

If on-chain tracking for connected spokes is not possible, the wiped-off liabilities should be snapshotted by the protocol before being set to zero.

Remediation Comment

SOLVED: The `bridgePrincipal` is now being emitted in the event.

Remediation Hash

https://github.com/m0-foundation/m-portal-lite/commit/f59bdea95011d2d2f1f204cd0a73580a002c5216

Halborn strongly recommends conducting a follow-up assessment of the project either within six months or immediately following any material changes to the codebase, whichever comes first. This approach is crucial for maintaining the project’s integrity and addressing potential vulnerabilities introduced by code modifications.